

Lab Sheet: Final Keyword, Polymorphism

Course: Introduction to Programming Language II (Java)

Topic: Final Keyword, Reference Type vs. Object Type, Dynamic Dispatch, Abstraction, Abstract Classes, Interfaces

Level: Beginner to Intermediate

Duration: 3 hours

Prepared by: Ashraful Islam Paran, Dept. of CSE, UIU

1. Objectives

By the end of this lab, students will be able to:

- Understand and apply the `final` keyword to variables, methods, and classes.
- Differentiate between reference type and object type in Java.
- Implement dynamic dispatch and runtime polymorphism.
- Use abstract classes and interfaces for abstraction.
- Resolve ambiguities like the diamond problem using interfaces.
- Practice coding with examples and exercises on these topics.

2. The final Keyword

The `final` keyword in Java is used to restrict modifications. It can be applied to variables (making them constants), methods (preventing overriding), and classes (preventing inheritance). **Key Points:**

- **Final Variables:** Constants that cannot be reassigned. Must be initialized at declaration or in a constructor (for non-static) or static block (for static).
- **Blank Final Variables:** Declared without initialization but must be assigned once in a constructor.
- **Static Blank Final Variables:** Assigned once in a static block.
- **Final Methods:** Cannot be overridden in subclasses.
- **Final Classes:** Cannot be extended.

3. Example 1: Final Variable

```
public class FinalVariableExample {  
    public static void main(String[] args) {  
        final int MAX = 100;  
        System.out.println("MAX: " + MAX);  
        // MAX = 200; //      Error  
    }  
}
```

Output:

MAX: 100

4. Example 2: Blank Final Variable

```
class BlankFinalExample {  
    final int speed; // blank final  
    BlankFinalExample(int s) {  
        speed = s; // must assign value here  
    }  
    void showSpeed() {  
        System.out.println("Speed: " + speed);  
    }  
    public static void main(String[] args) {  
        BlankFinalExample obj = new BlankFinalExample(90);  
        obj.showSpeed(); // Output: Speed: 90  
    }  
}
```

Output:

Speed: 90

5. Example 3: Static Blank Final Variable

```
class StaticBlankFinalExample {  
    static final int MAX;  
    static {  
        MAX = 500; // must initialize here  
    }  
    public static void main(String[] args) {  
        System.out.println("MAX: " + MAX); // Output: MAX: 500  
    }  
}
```

Output:

MAX: 500

6. Example 4: Final Method

```
class Parent {
    final void show() {
        System.out.println("Final method in Parent");
    }
}

class Child extends Parent {
    // void show() { } //      Error: cannot override final method
}

public class FinalMethodExample {
    public static void main(String[] args) {
        new Child().show(); // Output: Final method in Parent
    }
}
```

Output:

Final method in Parent

7. Example 5: Final Class

```
final class Vehicle {
    void run() {
        System.out.println("Vehicle is running");
    }
}

// class Car extends Vehicle {} //      Error

public class FinalClassExample {
    public static void main(String[] args) {
        Vehicle v = new Vehicle();
        v.run(); // Output: Vehicle is running
    }
}
```

Output:

Vehicle is running

Task 2.1 – Final Variable Practice (10 min)

- Create a class Constants with final variables for PI (3.14159) and GRAVITY (9.8). Write a main method to print them. Attempt to reassign one and note the error.

Sample Code:

```
public class Constants {  
    public static void main(String[] args) {  
        final double PI = 3.14159;  
        final double GRAVITY = 9.8;  
        System.out.println("PI: " + PI);  
        System.out.println("GRAVITY: " + GRAVITY);  
        // PI = 3.14; // Error  
    }  
}
```

Task 2.2 – Blank Final in Constructor (10 min)

- a. Create a class `Person` with a blank final `String` `name`. Initialize it in the constructor. Add a method to display the name.

Sample Code:

```
class Person {  
    final String name;  
    Person(String n) {  
        name = n;  
    }  
    void display() {  
        System.out.println("Name: " + name);  
    }  
    public static void main(String[] args) {  
        Person p = new Person("Alice");  
        p.display(); // Output: Name: Alice  
    }  
}
```

Task 2.3 – Static Blank Final (10 min)

- a. Create a class Config with a static blank final int PORT. Initialize it in a static block to 8080. Print it in main.

Sample Code:

```
class Config {  
    static final int PORT;  
    static {  
        PORT = 8080;  
    }  
    public static void main(String[] args) {  
        System.out.println("PORT: " + PORT); // Output: PORT: 8080  
    }  
}
```

Task 2.4 – Final Method and Class (10 min)

- a. Create a final class MathUtils with a final method add(int a, int b) that returns the sum. Try to extend the class and override the method (note errors).

Sample Code:

```
final class MathUtils {  
    final int add(int a, int b) {  
        return a + b;  
    }  
}  
  
// class ExtendedMath extends MathUtils {} //      Error  
  
public class MathTest {  
    public static void main(String[] args) {  
        MathUtils mu = new MathUtils();  
        System.out.println(mu.add(5, 3)); // Output: 8  
    }  
}
```


8. Reference Type vs. Object Type and Dynamic Dispatch

Reference type is the declared type of a variable (compile-time), while object type is the actual type in memory (runtime). Dynamic dispatch resolves overridden methods at runtime based on object type. **Key Points:**

- **Reference Type:** Determines accessible methods/fields at compile-time. From left side of assignment.
- **Object Type:** Determines which overridden methods run at runtime. From right side of assignment.
- **Dynamic Dispatch:** Overridden methods are called based on object type (runtime polymorphism).
- Variables and non-overridden methods are resolved by reference type.
- Fields are not polymorphic; resolved by reference type.
- Casting allows access to subclass members.

9. Example 6: Basic Reference vs Object Type

```
// Reference type: Parent
Parent p = new Child(); // Object type: Child
```

10. Example 7: Parent-Child Example

```
class Parent {
    int x = 10;
    void speak() {
        System.out.println("Parent speaks");
    }
}
class Child extends Parent {
    int x = 20;
    @Override
    void speak() {
        System.out.println("Child speaks");
    }
    void onlyChildDoes() {
        System.out.println("Child-only method");
    }
}
public class Test {
    public static void main(String[] args) {
        Parent p = new Child(); // ref: Parent, obj: Child
        System.out.println(p.x); // 10 (reference type wins)
        p.speak(); // Child speaks (object type wins)
        // p.onlyChildDoes(); // Compile error (Parent doesn't have
        //    ↪ this method)
        ((Child) p).onlyChildDoes(); // OK after casting
    }
}
```

Output:

10

Child speaks

Child-only method

11. Example 8: Animal-Dog-Cat Example

```
class Animal {
    String name = "Animal";
    void sound() {
        System.out.println("Animal makes a sound");
    }
}
class Dog extends Animal {
    String name = "Dog";
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}
class Cat extends Animal {
    String name = "Cat";
    @Override
    void sound() {
        System.out.println("Cat meows");
    }
}
public class Main {
    public static void main(String[] args) {
        Animal a1 = new Dog(); // parent reference, child object
        Animal a2 = new Cat(); // parent reference, child object
        Animal a3 = new Animal(); // parent reference, parent object
        // --- Dynamic Dispatch in action ---
        a1.sound(); // Dog barks
        a2.sound(); // Cat meows
        a3.sound(); // Animal makes a sound
        // --- Variable behavior ---
        System.out.println(a1.name); // Animal
        System.out.println(a2.name); // Animal
        System.out.println(a3.name); // Animal
    }
}
```

Output:

```
Dog barks
Cat meows
Animal makes a sound
Animal
Animal
Animal
```

Task 3.1 – Reference vs Object Type (12 min)

- Create classes `Vehicle` (with `int speed = 50`, `void move()`) and `Car` extends `Vehicle` (`speed = 100`, override `move()`). Use `Vehicle ref = new Car()` and print `speed/move`.

Sample Code:

```
class Vehicle {
    int speed = 50;
    void move() {
        System.out.println("Vehicle moves");
    }
}
class Car extends Vehicle {
    int speed = 100;
    @Override
    void move() {
        System.out.println("Car drives");
    }
}
public class VehicleTest {
    public static void main(String[] args) {
        Vehicle v = new Car();
        System.out.println(v.speed); // 50 (reference type)
        v.move(); // Car drives (object type)
    }
}
```

Task 3.2 – Dynamic Dispatch with Casting (12 min)

- a. Add a method honk () to Car only. Use casting to call it from Vehicle reference.

Sample Code:

```
// Extend from above
class Car extends Vehicle {
    // ... (previous)
    void honk() {
        System.out.println("Car honks");
    }
}

public class VehicleTest {
    public static void main(String[] args) {
        Vehicle v = new Car();
        // v.honk(); //      Compile error
        ((Car) v).honk(); // Car honks
    }
}
```

Task 3.3 – Multiple Subclasses (12 min)

- a. Create Bird extends Animal (name = "Bird", override sound() to "Bird chirps"). Use Animal refs for Dog, Cat, Bird and call sound()/print name.

Sample Code:

```
// Extend from Animal example
class Bird extends Animal {
    String name = "Bird";
    @Override
    void sound() {
        System.out.println("Bird chirps");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal a4 = new Bird();
        a4.sound(); // Bird chirps
        System.out.println(a4.name); // Animal
    }
}
```

Keep Coding!